AMATH 582: Home Work 5

Mengze Yu, Email address: mengzy3@uw.edu

This report is devoted to test the ability of FCN method and CNN method to classify the different figures from the FashionMNIST dataset. What's more, the report will also show the effect of different numbers of weights by changing the arguments when constructing layers. The other parameters and hyperparameters will keep the same as a model designed before.

Introduction and Overview

FashionMNIST dataset is a very common data set in machine learning. It includes 10 kinds of different figures which are saved as 28 × 28 images. Therefore, the shape of its feature is 28*28. We choose 90% figures' features and labels in the dataset as the training set and the figures' features and labels in the rest of dataset as testing sets randomly. The ultimate goal of the report is to use the FCN (Fully Convolutional Network) method and CNN (Convolutional Neural Networks) method to predict the label of a figure through its feature and the effect will be evaluated by the training loss, validation accuracy and the testing accuracy. We are also interested in effect of different number of weights when constructing layers.

Theoretical Background

One of the main methods used in the report is the FCN (Fully Convolutional Network) method. Its main propose is to construct hidden layers which includes different number of neurons and then transform the initial data, i.e. the features to the first layer, then the data will be passed through the middle layers and finally reaches the final layer, which is expected to be the labels. Despite the parameters and hyper parameters discussed before, the number of weights is also one of the most effective variables. It can be calculated as follow:

For any connection (hidden) layer we constructed, if its input dimension is I and its output dimension is O. Then the number of weights is $I \times O$, and its bias is O. The number of parameters generated is then the sum of the weight and the bias, i.e. $I \times O + O$. For instance, if our first connection layer is 28*28→124, then the weight is 784*124=97216 and the bias is 124. Thus, the number of parameters is 97340. The process also shows that we can use the result of the number of parameters in the model to approximate the weight since bias is always small.

What's more, another important method we used in the report is the CNN (Convolutional Neural Networks) method. The core idea of the method is to extract local features through convolution operations and reduce data dimensionality through pooling operations, thereby progressively extracting high-level feature representations. The convolution operation is a process sliding a convolutional kernel (filter) over the input data and computing the weighted sum of local regions. The formula is as follows:

$$(f * g) * (x, y) = \sum_i \sum_j f(i, j) \cdot g(x - i, y - j)$$

Where f is the input data (e.g., an image). g is the convolutional kernel. (x, y) is the position in the output feature map. Then we can apply a pooling operation to it to reduce the spatial dimensions of the feature map. By repeating these operations, a Fully Connected Layer can be constructed: $y = w \cdot x + b$. Where w is the weight matrix and b is the bias term. It's clear that the number of weights and bias in Connection Layer can be easily calculated with the same way in FCN method. However, a new formula should be introduced to calculate the number of parameters in Convolutional Layer as follow:

Suppose that our number of input channels is I, the number of output channels is O, the height of the kernel is H, and the width of the kernel is W. Then the number of weights is $I \times H \times W \times O$, while the bias is O. Then, the number of parameters in Convolutional Layer is the sum of the number of weights and bias and the total number of parameters is the sum of the number of parameters in each layer, including the Connection Layer and the Convolutional Layer. Similarly, we can also use the total number of parameters to approximate the weights, since bias is small.

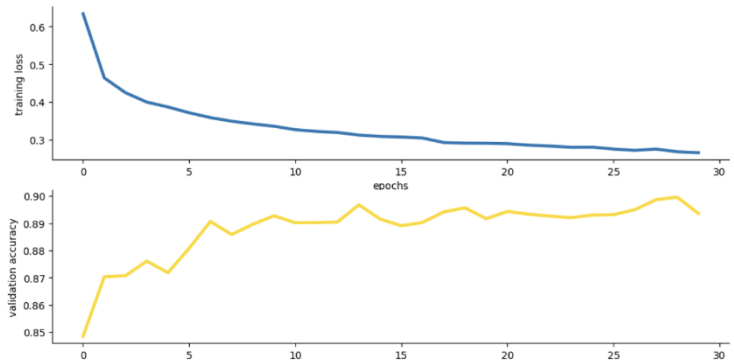
Algorithm Implementation and Development

In this report, numpy package is used for loading and processing the data, while the matplotlib package is used for visualizing the digit images. Beyond that, the sklearn package provides a convenient way to divide the training set and testing set. What's more, the pytorch package is also important for the report, since it allows us to download the dataset directly and is helpful to construct the network. The pytorch package can allow us to change the number of parameters.

Computational Results

Firstly, we need to download the dataset by pytorch and divide it into the training set and the testing set. Our strategy is to take 90% of data, including their features and labels, randomly as the training set and let the rest of the dataset be the testing set.

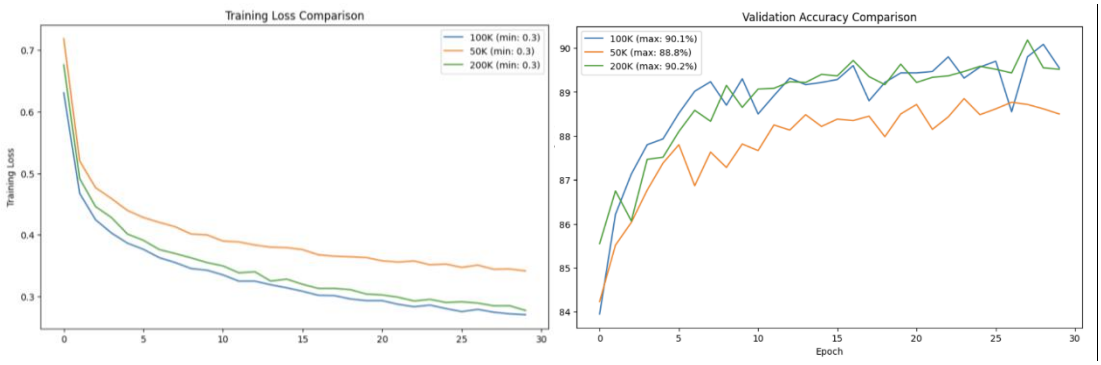
Then, we tried to construct our first FCN model, whose weight is less than 100K, we set the hidden layer as 124. Since the dimension of feature is 28*28 and the dimension of label is 10, the first connection layer is 28*28→124, the number of weights is 28*28*124=97216. The number of biases is 124. The second connection layer is 124→10. The number of weights is 124*10=1240, the number of biases is 10. It can be checked by the number of parameters in the model, which is about 98.6K. By using the setting in the last report, we used the Adam optimizer and set the learning rate as 0.01. What's more, we choose the Kaiming method as initial method. To train and test the model constructed, the number of the training batches is set as 512 and the number of epochs is 30, by the way, for the evaluation, we chose 256 as the number of testing batches. Also we used the dropout strategy and Batch normalization, the dropout rate is set as 0.5. Here is the result for the model (FCN 100K):



The figure of training loss and validation accuracy at each epoch

We choose the model which has the highest validation accuracy as the best model. The highest validation accuracy is 89.96% and the testing accuracy is 88.20%. The running time of the entire program is about 4 minutes, which is a reasonable time. To draw a conclusion, the validation accuracy and the testing accuracy are higher than 88%, which is acceptable, and nearly no overfitting or underfitting exists.
Thus, these parameters and hyper parameters will be our baseline parameters for the following steps.

Then, we tried to construct the FCN model whose number of weights is less than 50K and 200K. For the 50K model, we changed the hidden layer to 62, then the first connection layer is 28*28→62, the number of weights is 28*28*62=48608. The number of biases is 62. The second connection layer is 62→10. The number of weights is 62*10=620, the number of biases is 10. It can be checked by the number of parameters in the model, which is about 49.3K. Similarly, for the 50K model, we changed the hidden layer to 200. Then the first connection layer is 28*28→200, the second connection layer is 200→10. The total number of parameters in the model is about 178.1K. Here are some results and comparison about the models with different number of weights:



The figure of training loss & validation accuracy at each epoch with different weights

Also, we can compare the minimum training loss, maximum validation accuracy, test accuracy and processing time of each learning rate:

variant	Minimum Training loss	Maximum Validation accuracy	Testing accuracy	Processing time
FCN (100K)	0.2704	90.08%	88.08%	220s
FCN (50K)	0.2414	88.85%	87.04%	217s
FCN (200K)	0.2772	90.18%	88.35%	220s

minimum training loss, maximum validation accuracy, test accuracy and processing time with different number of weights of FCN model

To draw a conclusion, the number of weights seems to have some effects on the accuracy, especially, when the number of weights is small, the validation accuracy and the testing accuracy is losing visibly. However, the processing time is only affected slightly when we reduce the number of parameters. Moreover, the result of model FCN (100K) is similar to model FCN (200K). They have the same processing time, while the accuracy model FCN (200K) is higher slightly. Therefore, to construct a model which has a higher number of parameters is a better choice for FCN method.

The next part of this report is to construct CNN models for further comparison. Our basic model for CNN is the CNN 100K variant, whose number of weights is less than 100. To ensure the condition, we set the layers as follows: Firstly, we construct two convolution layers, for the first layer, we have 1 input channel and 32 output channels(kernel), the size of our kernel is 3×3. Therefore, the number of weights is 1*3*3*32=288, and the bias is 32. For the second layer, we have 32 input channel and 48 output channels(kernel), the size of our kernel is 3×3. Therefore, the number of weights is 32*3*3*48=13824, and the bias is 48. Also, after each step, we used a 2×2 maximum pool for extraction. It's obvious that the result, which is the input for the connection layer is 48*7*7 and we construct a hidden layer as 35. Then the first connection layer is 48*7*7→35, the number of weights is 48*7*7*35=82320. The number of biases is 35. The second connection layer is 35→10. The number of weights is 35*10=350, the number of biases is 10.

Therefore, the total number of parameters is 96907, which can be checked. Similar to the FCN models, we've also used Adam optimizer and set the learning rate as 0.01. What's more, the Kaiming method is chosen as initial method. To train and test the model constructed, the number of the training batches is set as 512 and the number of epochs is 30, by the way, for the evaluation, we chose 256 as the number of testing batches. Also, we used the dropout strategy with the dropout rate 0.5 on the connection layer and Batch normalization. Here's the result of the CNN model:

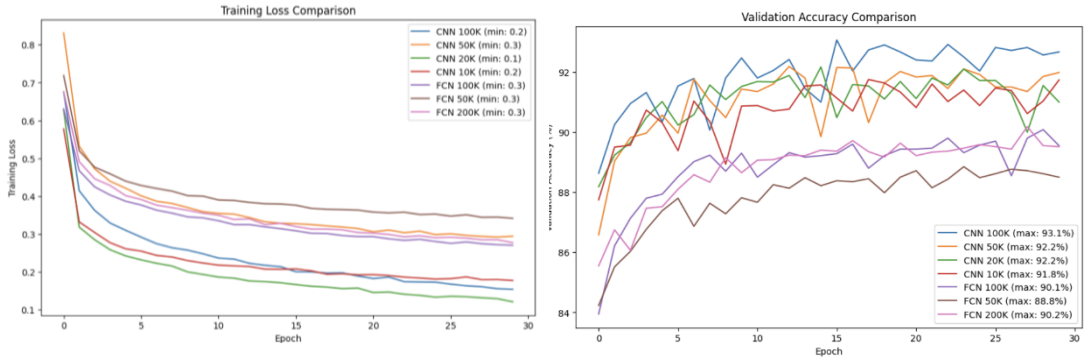
It takes 292 seconds to complete the whole process. The maximum validation accuracy is 93.07%, while the testing accuracy is 91.98%. That's a good result which ensures high validation accuracy and testing accuracy, while no overfitting or underfitting happens. Therefore, it will become our basic model of CNN method. What's more, it's obvious that although the CNN method takes more time, it has higher accuracies than FCN.

Next, we construct some other variants of CNN methods by reducing the number of parameters. For the CNN (50K) model, we also construct two convolution layers, for the first layer, we have 1 input channel and 16 output channels(kernel), the size of our kernel is 3×3 and for the second layer, we have 16input channel and 24 output channels(kernel), the size of our kernel is 3×3. After each step, we used a 2×2 maximum pool for extraction. What's more for the connection layer, we have 24*7*7→24→10. The number of parameters is about 32.3K.

For the CNN (20K) model, we constructed the convolution layers the same as the CNN (50K) model, but changed the connection layer to 24*7*7→10 The number of parameters is then about 15.5K.

For the CNN (10K) model, we constructed the first convolution layers by 1 input channel and 8 output channels and the first convolution layers by 8 input channels and 16 output channels. The size of kernel and the pool keeps the same as before. The connection layer is then 16*7*7→10. The number of parameters is about 9.1K.

Here are some results for the different variants:



The figure of training loss & validation accuracy at each epoch for different variants of FCN method and CNN method

variant	Minimum Training loss	Maximum Validation accuracy	Testing accuracy	Processing time
FCN (100K)	0.2704	90.08%	88.08%	220s
FCN (50K)	0.2414	88.85%	87.04%	217s
FCN (200K)	0.2772	90.18%	88.35%	220s
CNN (100K)	0.1539	93.07%	91.98%	292s
CNN (50K)	0.2941	92.18%	90.93%	281s
CNN (20K)	0.1209	92.17%	90.94%	285s
CNN (10K)	0.1772	91.75%	90.37%	282s

minimum training loss, maximum validation accuracy, test accuracy and processing time for different variants of FCN and CNN

It's super clear that even the CNN (10K) model will have better validation accuracy, testing accuracy and lower training loss than the FCN variants, which shows that CNN method is much more effective than the FCN method when processing the image classification problems. However, to compare between the CNN methods, we can find some other interesting results. Generally speaking, the CNN (100K) model is similar with the CNN (50K) model since they both have two connection layers. However, the CNN (50K) model can save some processing time on the cost of some loss of accuracy, which is acceptable. The solution also works for the CNN (20K) method and CNN (10K) method. What's more, the CNN (50K) model and the CNN (20K) model show that we can construct two models which have similar effect but with different number of parameters, by changing the connection layer. Thus, we can draw a conclusion that for the CNN method we can use some model with smaller weights and process faster but have lower accuracies.

Summary and Conclusions

To draw a conclusion, we succeeded in constructing several variants of FCN models and CNN models. They both showed good accuracies on processing the image classification problems, although the CNN method seems to be more effective. What's more, the number of weights seems to be important, since it will affect the accuracies. For the FCN method, the high number of weights will lead to better accuracies and will only raise a slight increasement of processing time. While for the CNN method, the effect of weights is more complex. We can construct some variants which have the similar effect but different number of weights. Also, we can construct a model which can save some processing time on the cost of some accuracies by reducing the number of weights.

Acknowledgements

The author is thankful to Prof. Eli Shlizerman for the examples in class, which helped to design the Network programs.